

Algoritmos de Optimización — Seminario

Combinar cifras y operaciones

Actividad grupal

Integrantes del grupo

- **Juan Alfonso Suyo Sullca**
- **Sebastián Alfredo Panesso Laverde**

✓ Algoritmos de optimización - Seminario

Nombre y Apellidos: Juan Alfonso Suyo Sullca · Sebastián Alfredo Panesso Laverde

Url: *(pega aquí el link a la carpeta de tu repositorio de GitHub con este notebook)*

Problema elegido:

- ~~1. Sesiones de doblaje~~
- ~~2. Organizar los horarios de partidos de La Liga~~
- 3. Combinar cifras y operaciones**

Descripción del problema (enunciado original):

El problema consiste en analizar el siguiente problema y diseñar un algoritmo que lo resuelva.

Disponemos de las 9 cifras del 1 al 9 (excluimos el cero) y de los 4 signos básicos de las operaciones fundamentales: suma (+), resta (-), multiplicación (*) y división (/).

Debemos combinarlos alternativamente sin repetir ninguno de ellos para obtener una cantidad dada. Un ejemplo sería para obtener el 4:

$$4+2-6/3*1 = 4$$

Debe analizarse el problema para encontrar todos los valores enteros posibles planteando las siguientes cuestiones:

- ¿Qué valor máximo y mínimo se pueden obtener según las condiciones del problema?
- ¿Es posible encontrar todos los valores enteros posibles entre dicho mínimo y máximo?

Nota: Es posible usar la función de Python `eval` para evaluar una expresión.

(*) La respuesta es obligatoria

```
# Librerías y datos base del problema
import random
import time
from math import factorial

CIFRAS = list(range(1, 10))          # 1,2,...,9 (excluimos el 0, como dice
OPERADORES = ['+', '-', '*', '/']   # los 4 signos, cada uno se usa una sol

print("Cifras disponibles:", CIFRAS)
print("Operadores disponibles:", OPERADORES)

Cifras disponibles: [1, 2, 3, 4, 5, 6, 7, 8, 9]
Operadores disponibles: ['+', '-', '*', '/']
```

(*)¿Cuántas posibilidades hay sin tener en cuenta las restricciones?

¿Cuántas posibilidades hay teniendo en cuenta todas las restricciones.

Para poder combinar las cifras con los operadores, primero tengo que dejar clara la "forma" que va a tener la expresión. Si me fijo en el ejemplo del enunciado (`4+2-6/3*1`), veo que se usan **5 cifras** y **4 operadores** alternados: cifra-operador-cifra-operador-cifra-operador-cifra-operador-cifra. Tiene sentido, porque solo hay 4 signos distintos (+, -, *, /) y si no se pueden repetir, la expresión más larga que puedo armar alternando cifra y operador queda en 5 cifras y 4 operadores.

Entonces el "tamaño" de cada expresión está fijo: elijo 5 de las 9 cifras disponibles (en un orden concreto) y los 4 operadores (también en un orden concreto).

Sin tener en cuenta las restricciones (o sea, permitiendo repetir cifras y operadores, como si cada posición fuera totalmente independiente de las demás), estoy en el caso de **variaciones con repetición** que vimos en el resumen de combinatoria:

- Para las 5 posiciones de cifra, cada una puede ser cualquiera de las 9 cifras:
 $VR_9^5 = 9^5 = 59.049$
- Para las 4 posiciones de operador, cada una puede ser cualquiera de los 4 signos: $VR_4^4 = 4^4 = 256$
- Total sin restricciones: $9^5 \times 4^4 = 15.116.544$ combinaciones posibles.

Teniendo en cuenta todas las restricciones (no se puede repetir ni cifra ni operador), ya es un caso de **variaciones sin repetición** (para las cifras) y **permutaciones** (para los operadores, porque uso los 4 sin excepción):

- Para las cifras: $V_9^5 = 9 \cdot 8 \cdot 7 \cdot 6 \cdot 5 = 15.120$
- Para los operadores: $P_4 = 4! = 24$
- Total con restricciones: $15.120 \times 24 = 362.880$ combinaciones válidas (que además coincide con $9! = 362.880$; tiene sentido porque $V_9^5 \times 4! = \frac{9!}{4!} \times 4! = 9!$).

O sea que, de las más de 15 millones de combinaciones "ingenuas", solo una pequeña fracción son realmente válidas:

$$\frac{362.880}{15.116.544} = 0.024 \rightarrow 2.4\%$$

Esto ya me da una pista de que hacer fuerza bruta de la forma más simple va a perder muchísimo tiempo generando cosas que ni siquiera cumplen las reglas del problema (el 97.6% restante se descarta después de haberlo generado).

```
# Verifico los cálculos de combinatoria con código

sin_restricciones = 9**5 * 4**4
con_restricciones = (factorial(9) // factorial(4)) * factorial(4)

print(f"Combinaciones SIN restricciones (con repetición): {sin_restricciones}")
print(f"Combinaciones CON restricciones (sin repetición): {con_restricciones}")
print(f"Eso es lo mismo que 9! = {factorial(9):,}")
print(f"Porcentaje de combinaciones válidas: {con_restricciones / sin_restricciones * 100}%")
```

```
Combinaciones SIN restricciones (con repetición): 15,116,544
Combinaciones CON restricciones (sin repetición): 362,880
Eso es lo mismo que 9! = 362,880
Porcentaje de combinaciones válidas: 2.40%
```

Modelo para el espacio de soluciones

(*) ¿Cual es la estructura de datos que mejor se adapta al problema? Argumentalo. (Es posible que hayas elegido una al principio y veas la necesidad de cambiar, argumentalo)

Para este problema decidí usar **listas** de Python trabajando juntas:

- `cifras_disponibles` y `operadores_disponibles`: son los elementos que todavía no he usado. Empiezan con las 9 cifras y los 4 operadores completos.
- `cifras_elegidas` y `operadores_elegidas`: es la expresión que voy construyendo poco a poco, en orden (acá el orden sí importa).

Según el modelo para el espacio de soluciones

(*) ¿Cual es la función objetivo?

(*) ¿Es un problema de maximización o minimización?

La función objetivo acá es bastante directa: dada una expresión completa (por ejemplo `4+2-6/3*1`), la evalúo con `eval()` y ese número es el resultado.

$$f(\text{expresión}) = \text{eval}(\text{expresión})$$

Lo particular de este problema es que **no es una única optimización**, sino en realidad dos (más una tercera tarea que ya no es optimización pura):

1. **Maximizar** f : encontrar la expresión que da el resultado más grande posible.
2. **Minimizar** f : encontrar la expresión que da el resultado más chico posible (o sea, más negativo).
3. Una vez que tengo el mínimo y el máximo, **enumerar** todos los valores enteros que hay entre ellos y comprobar si de verdad se pueden alcanzar todos o si quedan "huecos". Esto último ya no es optimizar, es más bien un problema de búsqueda/cobertura exhaustiva sobre el mismo espacio de soluciones.

Como maximizar y minimizar no compiten entre sí (no hay que sacrificar uno para lograr el otro, son dos búsquedas independientes sobre el mismo conjunto de expresiones posibles), esto no es un problema multiobjetivo con conflicto como el ejemplo de tiempo/combustible que vimos en clase. Resuelvo las dos cosas con el mismo barrido de todas las expresiones válidas.

```
# Pruebo con el ejemplo que trae el enunciado
ejemplo = "4+2-6/3*1"
print(f"eval({ejemplo}) = {eval(ejemplo)}")

eval(4+2-6/3*1) = 4.0
```

Diseña un algoritmo para resolver el problema por fuerza bruta

Mi primera idea, la más "obvia", es armar la expresión con simples bucles `for` anidados: uno para cada posición de cifra y uno para cada posición de operador, sin preocuparme al principio por si se repiten cifras u operadores. O sea, primero genero **todas** las combinaciones posibles (incluso las que no cumplen las reglas) y solo al final, cuando ya tengo la expresión completa armada, reviso si es válida (que no se repita ninguna cifra ni ningún operador) y, si lo es, la evalúo.

Esto es fuerza bruta en el sentido más literal: probar todo lo que se pueda probar sin ser inteligente para descartar nada antes de tiempo. Como calculé en la pregunta de combinatoria, esto corresponde a recorrer las $9^5 \times 4^4 = 15.116.544$ combinaciones "sin restricciones", aunque al final solo 362.880 sean válidas.

```
def fuerza_bruta(cifras, operadores):
    """
```

```

Genera TODAS las combinaciones posibles (con repetición) de 5 cifras y 4
y solo evalúa las que resultan válidas (sin cifras ni operadores repetid
Devuelve un diccionario {valor_resultado: expresion_que_lo_logra}.
"""

resultados = {}
total_generadas = 0
total_validas = 0

for c0 in cifras:
    for c1 in cifras:
        for c2 in cifras:
            for c3 in cifras:
                for c4 in cifras:
                    cifras_expr = (c0, c1, c2, c3, c4)
                    for o0 in operadores:
                        for o1 in operadores:
                            for o2 in operadores:
                                for o3 in operadores:
                                    total_generadas += 1
                                    operadores_expr = (o0, o1, o2, o3)

                                    # Recién ACÁ reviso si es válida (fu
                                    if len(set(cifras_expr)) != 5 or len
                                        continue

                                    total_validas += 1
                                    expr = f"{c0}{o0}{c1}{o1}{c2}{o2}{c3
                                    try:
                                        valor = eval(expr)
                                    except ZeroDivisionError:
                                        continue
                                    if valor not in resultados:
                                        resultados[valor] = expr

    return resultados, total_generadas, total_validas

t_inicio = time.time()
resultados_fb, generadas_fb, validas_fb = fuerza_bruta(CIFRAS, OPERADORES)
tiempo_fb = time.time() - t_inicio

print(f"Tiempo de ejecución: {tiempo_fb:.2f} segundos")
print(f"Combinaciones generadas en total: {generadas_fb:,}")
print(f"Combinaciones válidas evaluadas: {validas_fb:,}")
print(f"Valores distintos encontrados: {len(resultados_fb)}")
print(f"Valor mínimo obtenido: {min(resultados_fb)}")
print(f"Valor máximo obtenido: {max(resultados_fb)}")

Tiempo de ejecución: 4.45 segundos
Combinaciones generadas en total: 15,116,544
Combinaciones válidas evaluadas: 362,880
Valores distintos encontrados: 4465
Valor mínimo obtenido: -70.71428571428571
Valor máximo obtenido: 78.83333333333333

```

Calcula la complejidad del algoritmo por fuerza bruta

Si cuento las operaciones elementales del algoritmo anterior:

- Tengo 5 bucles anidados recorriendo las $n = 9$ cifras cada uno (con repetición) $\Rightarrow n^5$ combinaciones de cifras.
- Por cada una, tengo 4 bucles anidados recorriendo los $m = 4$ operadores (con repetición) $\Rightarrow m^4$ combinaciones de operadores.
- Dentro del bucle más interno hago un trabajo de tamaño constante: armar dos conjuntos pequeños de tamaño fijo (5 y 4) para chequear repetidos, y si pasa la validación, armar un string de longitud fija y evaluarlo con `eval`. Todo eso es $O(1)$ porque no depende de n , siempre son 5 cifras y 4 operadores.

Entonces la complejidad total es:

$$O(n^5 \cdot m^4) = O(n^5)$$

(dejo $m = 4$ como constante porque el problema siempre va a tener exactamente esos 4 signos, eso no crece).

Con $n = 9$: $9^5 \cdot 4^4 = 15.116.544$ operaciones aproximadamente, que es justo lo que midió mi contador `generadas_fb` en el código de arriba. El tiempo real que tardó (podés verlo impreso arriba) fue de unos 4 segundos en mi computador; obviamente eso va a variar según el equipo donde se ejecute.

Es un algoritmo que en este caso puntual ($n = 9$) todavía es manejable, corre en pocos segundos, pero se nota clarísimo que está haciendo un montón de trabajo de más: de esas ~15 millones de combinaciones generadas, solo 362.880 (un 2.4%) terminan siendo válidas. El resto del tiempo se gasta armando y descartando cosas que ya sabíamos de antemano que no iban a servir.

```
n, m = 9, 4
formula_teorica = n**5 * m**4

print(f"Fórmula teórica n^5 * m^4 = {formula_teorica:,}")
print(f"Total generadas medido en el código = {generadas_fb:,}")
print("¿Coinciden?", formula_teorica == generadas_fb)
```

```
Fórmula teórica n^5 * m^4 = 15,116,544
Total generadas medido en el código = 15,116,544
¿Coinciden? True
```

(*)Diseña un algoritmo que mejore la complejidad del algoritmo por fuerza bruta. Argumenta porque crees que mejora el algoritmo por fuerza bruta

La fuerza bruta pierde muchísimo tiempo generando y descartando combinaciones que, si lo pienso bien, ya sé de antemano que van a fallar (las que repiten alguna cifra o algún operador). La técnica que vimos en clase para evitar justo esto es **vuelta atrás (backtracking)**: en vez de generar TODO y filtrar al final, voy construyendo la expresión un elemento a la vez, y en cada paso solo elijo entre los elementos que **todavía no he usado**. Así nunca llego a construir una combinación inválida, porque la propia estructura de datos (las listas de "disponibles") me lo impide desde el principio.

Es la misma idea que usamos en la Actividad Guiada 1 para las N-Reinas: en vez de generar todos los tableros posibles y después revisar cuáles son válidos, voy colocando reinas una por una y descarto (podo) la rama apenas veo que ya no puede funcionar.

¿Por qué mejora? Porque elimina por completo el trabajo de armar y evaluar las combinaciones inválidas, que eran el 97.6% del total en la fuerza bruta. La "función de descarte" acá es prácticamente gratis (sacar un elemento de una lista, $O(1)$), tal como se pedía en la teoría: la función de descarte debe ser tan sencilla como el costo de la exploración, si no, no mejora nada.

```
def generar_expresiones(
    cifras_disponibles,
    operadores_disponibles,
    cifras_elegidas,
    operadores_elegidos,
    resultados):
    """
    Construye la expresión con vuelta atrás: en cada paso solo elige cifras/
    que aún están disponibles, así que nunca genera una combinación inválida
    Va llenando 'resultados' con {valor: expresion_que_lo_logra}.
    """

    # Caso base: ya tengo las 5 cifras y los 4 operadores -> armo y evalúo
    if len(cifras_elegidas) == 5:
        expr = ""
        for i in range(4):
            expr += str(cifras_elegidas[i]) + operadores_elegidos[i]
        expr += str(cifras_elegidas[4])
        try:
            valor = eval(expr)
        except ZeroDivisionError:
            return
        if valor not in resultados:
            resultados[valor] = expr
        return

    # Etapa: elijo la siguiente cifra de las que quedan disponibles
    for i in range(len(cifras_disponibles)):
        cifra = cifras_disponibles[i]
        cifras_elegidas.append(cifra)
```

```

del cifras_disponibles[i]

if len(cifras_elegidas) == 1:
    # La primera cifra todavía no necesita operador antes
    generar_expresiones(cifras_disponibles, operadores_disponibles,
                        cifras_elegidas, operadores_elegidos, resul
else:
    # A partir de la segunda cifra, antes tengo que elegir un operad
    for j in range(len(operadores_disponibles)):
        operador = operadores_disponibles[j]
        operadores_elegidos.append(operador)
        del operadores_disponibles[j]

        generar_expresiones(cifras_disponibles, operadores_disponibl
                            cifras_elegidas, operadores_elegidos, r

    # Vuelta atrás: devuelvo el operador a la bolsa de disponibl
    operadores_disponibles.insert(j, operador)
    operadores_elegidos.pop()

# Vuelta atrás: devuelvo la cifra a la bolsa de disponibles
cifras_disponibles.insert(i, cifra)
cifras_elegidas.pop()

t_inicio = time.time()
resultados_bt = {}
generar_expresiones(CIFRAS.copy(), OPERADORES.copy(), [], [], resultados_bt)
tiempo_bt = time.time() - t_inicio

print(f"Tiempo de ejecución: {tiempo_bt:.2f} segundos")
print(f"Valores distintos encontrados: {len(resultados_bt)}")
print(f"Valor mínimo obtenido: {min(resultados_bt)}")
print(f"Valor máximo obtenido: {max(resultados_bt)}")

Tiempo de ejecución: 1.98 segundos
Valores distintos encontrados: 4465
Valor mínimo obtenido: -70.71428571428571
Valor máximo obtenido: 78.83333333333333

```

(*)Calcula la complejidad del algoritmo

Con backtracking, el árbol de búsqueda solo genera ramas válidas. El número de expresiones completas que realmente construyo y evalúo es:

$$V_n^5 \cdot P_4 = n(n-1)(n-2)(n-3)(n-4) \cdot 4!$$

Que sigue siendo, en el peor caso, del orden $O(n^5)$ (el producto de 5 términos consecutivos que decrecen desde n es un polinomio de grado 5 en n), pero con una constante bastante más chica: para $n = 9$ obtengo $9 \cdot 8 \cdot 7 \cdot 6 \cdot 5 \cdot 24 = 362.880$, en vez de los 15.116.544 de la fuerza bruta. Eso es una reducción de aproximadamente **41.6 veces** menos combinaciones evaluadas.

Curiosamente, cuando medí el tiempo real de ejecución, la mejora no fue de 41.6x sino de apenas 2.2x (fuerza bruta ronda los 4 segundos, backtracking ronda los 2 segundos, podés comparar los tiempos impresos arriba). Al principio pensé que algo estaba mal, pero tiene sentido: la mayoría de las "iteraciones de más" que hace la fuerza bruta son bien baratas (solo comprueban si hay repetidos con un `set()` de tamaño fijo y siguen de largo con `continue`), nunca llegan a construir el string ni a llamar `eval()`, que es lo realmente costoso. O sea que el ahorro *combinatorio* (cuántas combinaciones válidas se evalúan) es mucho más grande que el ahorro *real en tiempo de reloj*, porque no todas las operaciones "ahorradas" pesaban lo mismo.

```
n = 9
combinaciones_fb = n**5 * 4**4
combinaciones_bt = n * (n-1) * (n-2) * (n-3) * (n-4) * factorial(4)

print("Comparación fuerza bruta vs. backtracking (n = 9)")
print(f"  Combinaciones evaluadas fuerza bruta:  {combinaciones_fb:,}")
print(f"  Combinaciones evaluadas backtracking:  {combinaciones_bt:,}")
print(f"  Reducción combinatoria: {combinaciones_fb / combinaciones_bt:.1f}x")
print()
print(f"  Tiempo real fuerza bruta:  {tiempo_fb:.2f} s")
print(f"  Tiempo real backtracking: {tiempo_bt:.2f} s")
print(f"  Reducción real en tiempo: {tiempo_fb / tiempo_bt:.1f}x")
```

```
Comparación fuerza bruta vs. backtracking (n = 9)
Combinaciones evaluadas fuerza bruta:  15,116,544
Combinaciones evaluadas backtracking:   362,880
Reducción combinatoria: 41.7x
```

```
Tiempo real fuerza bruta:  4.45 s
Tiempo real backtracking: 1.98 s
Reducción real en tiempo: 2.2x
```

Según el problema (y tenga sentido), diseña un juego de datos de entrada aleatorios

Acá tuve que pensar distinto a como lo haría con, por ejemplo, el problema de La Liga o el de doblaje, porque en este problema el "tamaño de entrada" está fijo: siempre son las mismas 9 cifras y los mismos 4 operadores, no hay una forma natural de agrandar o achicar esa parte de la entrada.

Lo que sí puede variar, y tiene sentido probar con datos aleatorios, es la **cantidad objetivo** que uno quiere alcanzar. Entonces genero una lista de números aleatorios (algunos dentro del rango que encontré que es alcanzable, y otros a propósito fuera de ese rango) y para cada uno reviso si mi algoritmo encuentra una expresión que lo logre.

```
random.seed(42) # fijo la semilla para que el resultado se pueda reproducir
```

```
# Genero 10 objetivos aleatorios en un rango bastante más amplio que el alca
# a propósito, para ver que el algoritmo también detecte bien cuando algo NO
objetivos_aleatorios = [random.randint(-90, 100) for _ in range(10)]
print("Objetivos generados aleatoriamente:", objetivos_aleatorios)
```

```
Objetivos generados aleatoriamente: [73, -62, -84, 99, -20, -28, -33, -55, 98
```

Aplica el algoritmo al juego de datos generado

Reutilizo `resultados_bt` (el diccionario que arma el backtracking) quedándome solo con los valores que son números enteros, y con eso reviso, uno por uno, si cada objetivo aleatorio tiene una expresión que lo logre. De paso, con este mismo diccionario respondo la pregunta del enunciado sobre el mínimo, el máximo y si están cubiertos todos los enteros intermedios.

```
# Me quedo solo con los resultados que son valores enteros (sin parte decima
enteros_alcanzables = {int(v): expr for v, expr in resultados_bt.items() if

minimo_entero = min(enteros_alcanzables)
maximo_entero = max(enteros_alcanzables)
rango_completo = set(range(minimo_entero, maximo_entero + 1))
huecos = sorted(rango_completo - set(enteros_alcanzables))

print(f"Valor entero mínimo alcanzable: {minimo_entero}")
print(f"Valor entero máximo alcanzable: {maximo_entero}")
print(f"Cantidad de enteros distintos alcanzables: {len(enteros_alcanzables)}")
print(f"Enteros del rango [{minimo_entero}, {maximo_entero}] que NO se puede
print()

print("--- Aplicando el algoritmo a los objetivos aleatorios ---")
for objetivo in objetivos_aleatorios:
    if objetivo in enteros_alcanzables:
        expr = enteros_alcanzables[objetivo]
        print(f" {objetivo:>4}: SI se puede lograr -> {expr} = {objetivo}")
    else:
        print(f" {objetivo:>4}: NO se puede lograr con este esquema de 5 ci
```

```
Valor entero mínimo alcanzable: -69
Valor entero máximo alcanzable: 77
Cantidad de enteros distintos alcanzables: 147
Enteros del rango [-69, 77] que NO se pueden lograr: ninguno, se cubren todos
```

```
--- Aplicando el algoritmo a los objetivos aleatorios ---
73: SI se puede lograr -> 3/1-2+8*9 = 73
-62: SI se puede lograr -> 3/1+7-8*9 = -62
-84: NO se puede lograr con este esquema de 5 cifras y 4 operadores
99: NO se puede lograr con este esquema de 5 cifras y 4 operadores
-20: SI se puede lograr -> 1+3-6/2*8 = -20
-28: SI se puede lograr -> 1-4*8+6/2 = -28
-33: SI se puede lograr -> 1-4*9+6/3 = -33
-55: SI se puede lograr -> 2/1+6-7*9 = -55
```

98: NO se puede lograr con este esquema de 5 cifras y 4 operadores
-64: SI se puede lograr -> $2/1+6-8*9 = -64$

Enumera las referencias que has utilizado(si ha sido necesario) para llevar a cabo el trabajo

- Material de clase de la asignatura *Algoritmos de Optimización* (VIU - 03MIAR): sesiones VC1 (Introducción a los algoritmos), VC2 (Diseño de algoritmos: vuelta atrás, divide y vencerás, programación dinámica) y la Actividad Guiada 1.
- Documento "Resumen de Combinatoria" entregado en la asignatura (fórmulas de variaciones, permutaciones y combinaciones con y sin repetición).
- Documentación oficial de Python para `eval()`:
<https://docs.python.org/3/library/functions.html#eval>
- Documentación oficial de Python para el módulo `random`:
<https://docs.python.org/3/library/random.html>
- Refactorización de función `generar_expresiones` con ClaudeCode.

Describe brevemente las líneas de como crees que es posible avanzar en el estudio del problema. Ten en cuenta incluso posibles variaciones del problema y/o variaciones al alza del tamaño

Como posible continuación de este trabajo, sería interesante analizar cómo se comporta el algoritmo cuando aumenta el tamaño del problema. Si se dispusiera de un mayor número de cifras u operadores, el número de combinaciones crecería rápidamente y la búsqueda por fuerza bruta dejaría de ser una opción eficiente. También se podrían incorporar estrategias que permitieran reducir el número de combinaciones que es necesario explorar, por ejemplo mediante podas durante la búsqueda o utilizando heurísticas para priorizar las alternativas con más posibilidades de encontrar una solución. Finalmente, si el tamaño del problema aumentara considerablemente, sería interesante estudiar el uso de algoritmos genéticos, para obtener buenas soluciones en un tiempo razonable sin necesidad de evaluar todas las combinaciones posibles.